Code Explanation

SCD Type 2 Updater Code Explanation

The provided code is a PySpark application designed to handle Slowly Changing Dimension (SCD) Type 2 updates for the `ordersummary` table when there are changes in the `customer` table. It ensures data consistency by updating the `ordersummary` table and subsequently updating the `customeraggregatespend` table.

Overview of the Code

This PySpark application performs SCD Type 2 updates on the `ordersummary` table based on changes in the `customer` table. It involves reading the current state of both tables, identifying records with customer changes, updating the `ordersummary` table accordingly, and then updating the `customeraggregatespend` table based on the new state of `ordersummary`.

Step 1: Reading Current Customer and Ordersummary Tables
The function `update_scd_type2` starts by reading the current `customer` and `ordersummary` tables from the specified catalog and schema using SparkSession.

```
customer_df = spark.table(f"{catalog}.{schema}.customer")
ordersummary_df = spark.table(f"{catalog}.{schema}.ordersummary").fi
lter("IsActive = true")
```

Step 2: Identifying Records with Customer Changes
It then joins the `ordersummary_df` with `customer_df` on `CustId` to identify records where customer information has changed.

```
changed_records = ordersummary_df.alias("o").join(
    customer_df.alias("c"),
    col("o.CustId") == col("c.CustId"),
    "inner"
).filter(
    (col("o.Name") != col("c.Name")) |
    (col("o.EmailId") != col("c.EmailId")) |
    (col("o.Region") != col("c.Region"))
).select("o.*")
```

Step 3: Updating SCD Type 2 Table for Changed Records
If there are changed records, it updates the existing records in the `ordersummary` Delta table to mark them as inactive and sets an `EndDate`.

```
delta_table = DeltaTable.forName(spark, f"{catalog}.{schema}.ordersu
mmary")
delta_table.alias("target").merge(
    changed_records.alias("source"),
    "target.CustId = source.CustId AND target.IsActive = true"
).whenMatched().updateExpr({
    "IsActive": "false",
    "EndDate": "current_timestamp()"
```

```
}).execute()
```

Step 4: Creating New Records with Updated Customer Information
New records are created with the updated customer information and inserted into the `ordersummary` table.

```
new_records = changed_records.join(
    customer_df,
    "CustId",
    "inner"
).select(
    customer_df["CustId"],
    customer_df["Name"],
    customer_df["EmailId"],
    customer_df["Region"],
    changed_records["OrderId"],
    changed_records["ItemName"],
    changed_records["PricePerUnit"],
    changed_records["Qty"],
    changed_records["Date"],
    changed_records["TotalAmount"],
    lit(True).alias("IsActive"),
    current_timestamp().alias("StartDate"),
    lit(None).cast(TimestampType()).alias("EndDate")
)
new_records.write.format("delta").mode("append").saveAsTable(f"{cata
log}.{schema}.ordersummary")
```

Step 5: Updating Customer Aggregate Spend Table
After updating the `ordersummary` table, the `customeraggregatespend` table is updated by aggregating the total spend by customer name and date from the active records in `ordersummary`.

```
def update_customer_aggregate_spend(spark, catalog, schema):
    ordersummary_df = spark.table(f"{catalog}.{schema}.ordersummary"
).filter("IsActive = true")
    aggregated_df = ordersummary_df.groupBy("Name", "Date")
                                   .agg({"TotalAmount": "sum"})
                                   .withColumnRenamed("sum(TotalAmoun
t)", "TotalAmount")
    aggregated_df.write.format("delta").mode("overwrite").saveAsTabl
e(f"{catalog}.{schema}.customeraggregatespend")
```

Step 6: Main Execution
The `main` function initializes a SparkSession, sets the catalog and schema configurations, and calls `update_scd_type2` to start the SCD Type 2 update process.

```
def main():
    spark = SparkSession.builder
        .appName("SCD Type 2 Updater")
        .getOrCreate()
```

```
    catalog = "gen_ai_poc_databrickscoe"
    schema = "sdlc_wizard"
    update_scd_type2(spark, catalog, schema)

if __name__ == "__main__":
    main()
```